INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

TUTORIAL: ACCESO A LA BASE DE DATOS CON MYSQL WORKBENCH

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Qué base de datos usa LeGrandHotel 3
1.1 Qué vas a lograr 3
1.2 Qué necesitas 3
2. Los datos de conexión 3
3. Paso 1: encender Docker Desktop 4
4. Paso 2: encender la base de datos 4
5. Paso 3: tener las tablas al día 5
6. Paso 4: abrir MySQL Workbench 6
7. Paso 5: crear la conexión 6
8. Paso 6: guardar la contraseña y probar 8
9. Paso 7: conectarte y ver las tablas 9
10. Qué guarda cada tabla 9
10.1 Tablas del hotel 9
10.2 Cuentas y permisos 10
10.3 Tablas internas de Laravel 10
11. Consultas útiles (con su resultado real) 10
11.1 Cuántos registros hay en cada tabla 11
11.2 Cuentas de demostración y su rol 11
11.3 Tipos de habitación y lo que incluyen 11
11.4 Próximas llegadas 12
11.5 Reservas por estado 12
12. Cuidado al modificar datos 13
13. Respaldo y apagado 13
13.1 Hacer un respaldo 13
13.2 Apagar la base 13
14. Si algo sale mal 14
15. Sin Workbench: consultar desde la terminal 14
16. Resumen rápido 15

1. Qué base de datos usa LeGrandHotel

LeGrandHotel guarda todo (huéspedes, reservas, habitaciones, inventario, cuentas y auditoría) en una base de datos MySQL 8.4 llamada legrandhotel. En la computadora de desarrollo esa base corre dentro de Docker; en la nube correrá en Amazon RDS.

Dónde	Motor	Para qué
Laptop (este tutorial)	MySQL 8.4 en Docker (contenedor legrandhotel-mysql-1)	Desarrollar, probar la página y revisar los datos
Pruebas automáticas (Pest)	SQLite en memoria	Se crea y se borra sola en cada prueba; no se abre con Workbench
AWS (entrega final)	MySQL en Amazon RDS	La base real del hotel

Por qué en Docker: Docker trae MySQL ya configurado, con la misma versión para todo el equipo, sin instalarlo en Windows. Los datos se guardan en un volumen de Docker (mysql-data), así que no se pierden al apagar el contenedor.

1.1 Qué vas a lograr

1. Encender la base de datos.
2. Asegurarte de que tiene todas sus tablas al día.
3. Conectarte con MySQL Workbench y guardar la conexión.
4. Ver las tablas, entender qué guarda cada una y hacer consultas.
5. Hacer un respaldo y apagar todo sin perder datos.

1.2 Qué necesitas

- Docker Desktop instalado.
- MySQL Workbench instalado (este tutorial usa la versión 26.10, la nueva interfaz de Workbench).
- El proyecto en C:\Proyectos\LeGrandHotel con su archivo api\.env.
- PHP 8.2 (solo para el paso 3, crear o actualizar las tablas).

2. Los datos de conexión

Son los mismos para Workbench, para la terminal y para la API. Están en el archivo api\.env del proyecto (líneas que empiezan con DB_):

Dato	Valor	Dónde se ve en api\.env	Por qué
Host	127.0.0.1	DB_HOST	La base corre en tu propia computadora. Se escribe 127.0.0.1 y no «localhost» para que Windows no intente IPv6
Puerto	3306	DB_PORT	Puerto estándar de MySQL; Docker lo conecta con el del contenedor

Dato	Valor	Dónde se ve en api\.env	Por qué
Usuario	legrandhotel	DB_USERNAME	Usuario propio del proyecto (no root): solo tiene permiso sobre la base legrandhotel
Contraseña	la de DB_PASSWORD	DB_PASSWORD	Debe ser igual a MYSQL_PASSWORD del archivo docker-compose.yml, que es con la que se creó la base
Base (esquema)	legrandhotel	DB_DATABASE	Es donde Laravel creó todas las tablas

Sobre la contraseña: ábrela en api\.env con el Bloc de notas o VS Code y cópiala de la línea DB_PASSWORD. No la escribas en documentos ni la subas a GitHub: el archivo .env está fuera del repositorio a propósito.

3. Paso 1: encender Docker Desktop

1. Abre Docker Desktop desde el menú Inicio.
2. Espera a que abajo a la izquierda diga «Engine running» (motor corriendo). La primera vez puede tardar uno o dos minutos.

Por qué: MySQL vive dentro de Docker. Si Docker está apagado, ningún programa (ni Workbench ni la API) puede llegar a la base, y verás el error «Can't connect to MySQL server».

4. Paso 2: encender la base de datos

Abre PowerShell en la carpeta del proyecto (la línea debe decir PS C:\Proyectos\LeGrandHotel>) y escribe:

PowerShell
```
cd C:\Proyectos\LeGrandHotel
docker compose up -d mysql
```



Figura 1. Docker enciende el contenedor de MySQL.

Por qué: docker compose lee el archivo docker-compose.yml, que está en la carpeta del proyecto (por eso hay que estar ahí). La palabra mysql enciende solo la base de datos: si encendieras todo el proyecto, el contenedor nginx ocuparía el puerto 8000 que usa la API cuando la corres con PHP. La opción -d deja el contenedor corriendo en segundo plano y te devuelve la terminal.

Comprueba que está listo:

PowerShell
```
docker compose ps mysql
```

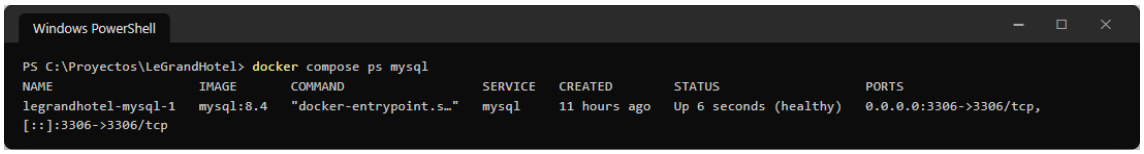


Figura 2. MySQL está encendido y sano (healthy) en el puerto 3306.

Qué debes ver: en la columna STATUS, «Up … (healthy)». Si dice «(health: starting)», espera unos segundos y repite el comando: MySQL todavía está arrancando. En PORTS debe aparecer 0.0.0.0:3306->3306/tcp, que significa que el puerto 3306 de tu computadora lleva al MySQL del contenedor.

5. Paso 3: tener las tablas al día

Laravel crea y actualiza las tablas con «migraciones». Hay que correrlas la primera vez y cada vez que alguien del equipo agrega una nueva (por ejemplo, cuando se agregó el rol Cliente):

PowerShell

```
cd api
php artisan migrate
```

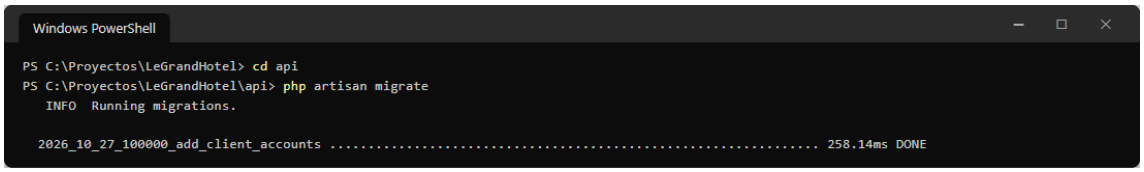


Figura 3. Laravel aplica la migración que faltaba.

Para ver cuáles están aplicadas:

PowerShell

```
php artisan migrate:status
```

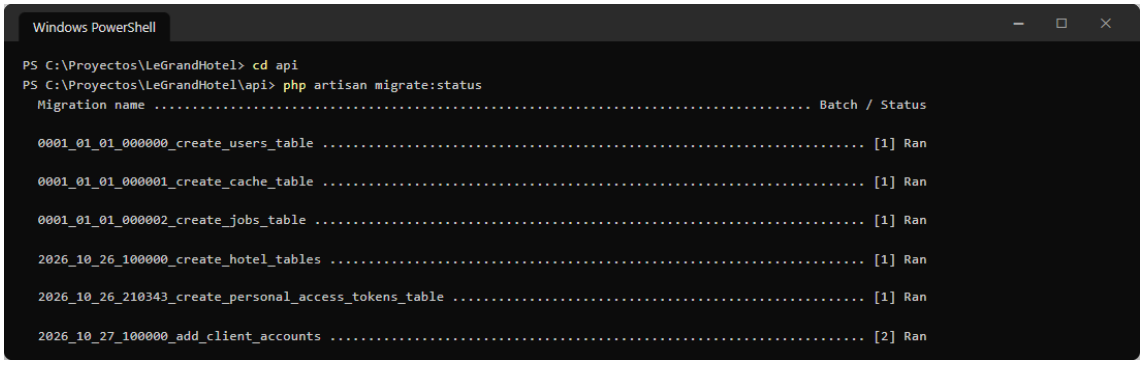


Figura 4. Todas las migraciones dicen «Ran»: la base está al día.

Por qué: si la base no tiene una migración reciente, le faltan tablas o columnas (por ejemplo clients.user_id o room_types.amenities) y la página falla al usarlas. Revisar el estado antes de abrir Workbench evita confundirse buscando una columna que todavía no existe. Las migraciones nunca borran datos existentes: solo agregan o cambian estructura.

Si la base está vacía (la primera vez), carga también los datos de demostración y los masivos:

PowerShell

```
php artisan db:seed --class=DemoSeeder
php artisan db:seed --class=MassiveDataSeeder
```

DemoSeeder crea las cuentas de prueba, los tipos y las habitaciones; MassiveDataSeeder agrega 1000 huéspedes y 5000 reservas. No corras MassiveDataSeeder dos veces: duplicaría los datos.

6. Paso 4: abrir MySQL Workbench

Ábrelo desde el menú Inicio. Verás la pantalla de bienvenida y, abajo, la sección Database Connections (conexiones guardadas). La primera vez está vacía.



Figura 5. Pantalla de inicio de MySQL Workbench 26.10. El botón + de la derecha crea una conexión.

Si la ventana se queda en negro: es la nueva interfaz cargando. Espera unos segundos; si sigue igual, pulsa Ctrl + B (mostrar u ocultar la barra lateral) para que se redibuje.

Por qué esta versión: Workbench 26.10 es la versión nueva y está hecha para MySQL 8.4 y posteriores, que es justo la versión que usamos en Docker. El Workbench 8.0 anterior muestra advertencias con MySQL 8.4.

7. Paso 5: crear la conexión

1. En la fila **Database Connections**, pulsa el botón **+** (a la derecha). Se abre la ventana Database Connection Configuration.

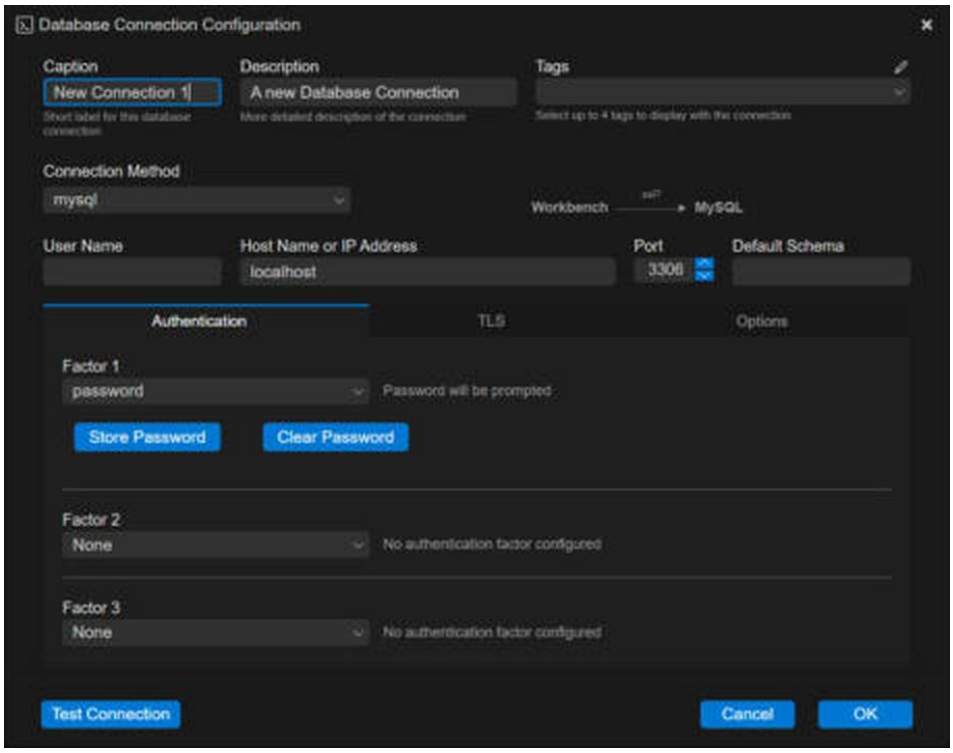


Figura 6. Ventana de nueva conexión, tal como aparece.

2. Llena los campos así:

Campo	**Qué escribir**	**Por qué**
Caption	LeGrandHotel (local)	Nombre con el que verás la conexión en la pantalla de inicio
Description	MySQL en Docker	Opcional: te recuerda de dónde es
Connection Method	mysql (déjalo así)	Conexión directa por TCP, que es como Docker expone la base
User Name	legrandhotel	El usuario del proyecto (DB_USERNAME)
Host Name or IP Address	127.0.0.1	Cambia «localhost» por 127.0.0.1 para usar IPv4, la dirección por donde Docker publica el puerto
Port	3306	Ya viene puesto
Default Schema	legrandhotel	Al conectarte, Workbench ya entra a esta base y no tienes que escribir USE legrandhotel
Pestañas TLS y Options	No cambies nada	En local no hace falta cifrar la conexión

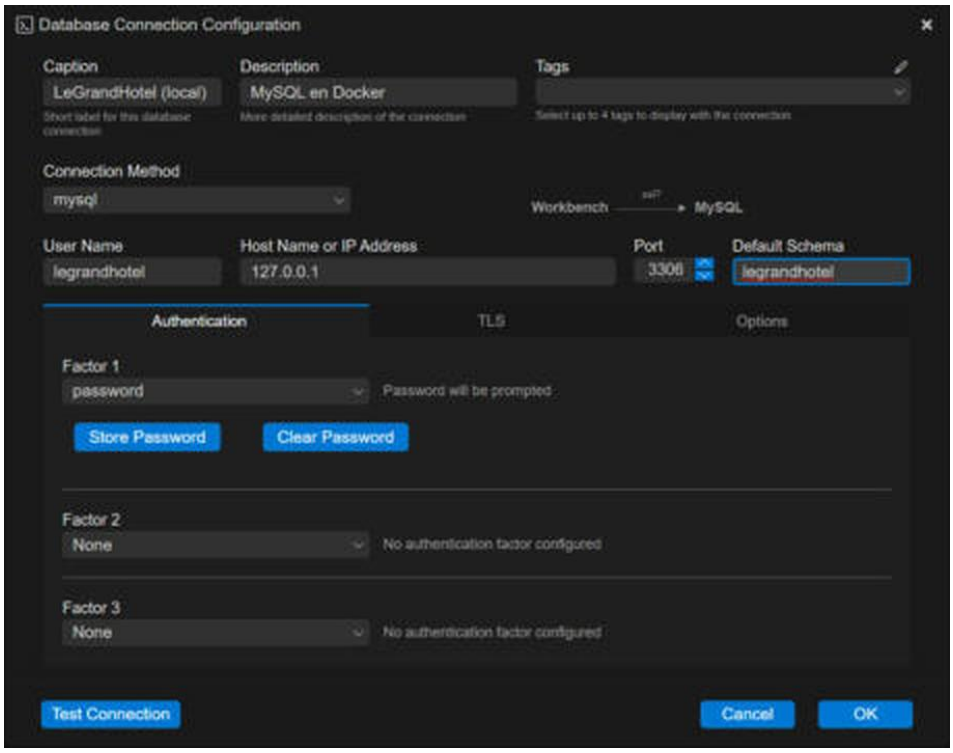


Figura 7. La conexión con los datos de LeGrandHotel.

8. Paso 6: guardar la contraseña y probar

1. En la pestaña **Authentication**, Factor 1 queda en «password». Pulsa **Store Password**.
2. Se abre el cuadro **Open MySQL Connection**. Escribe la contraseña (la de DB_PASSWORD en api\.env) y pulsa **OK**. Mientras escribes no se ve nada: es normal.

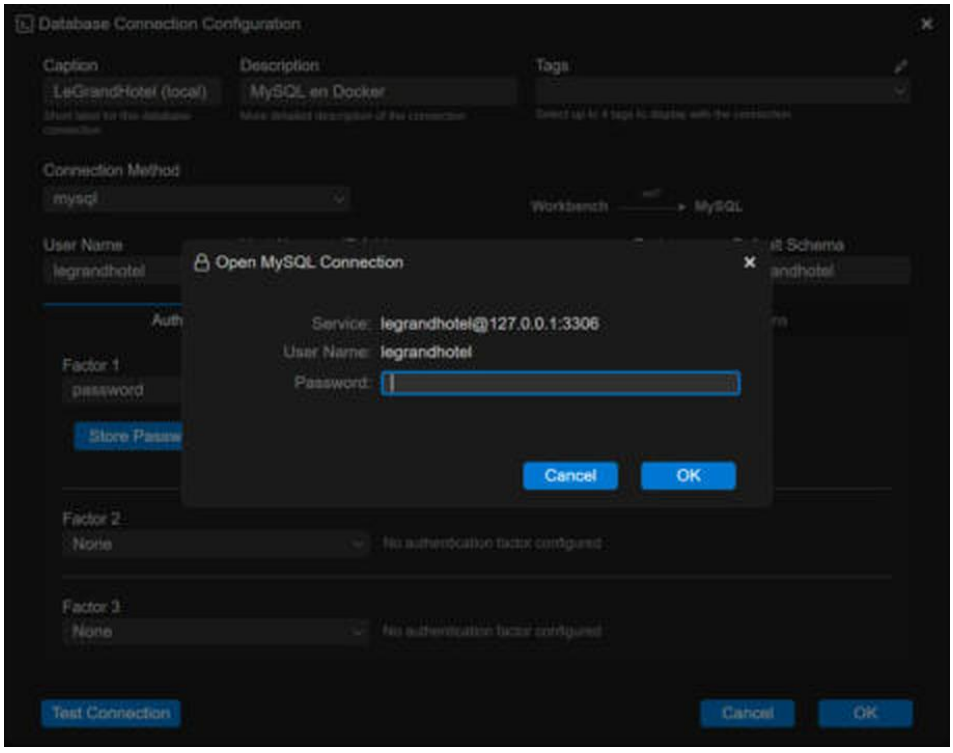


Figura 8. Cuadro para guardar la contraseña de forma segura.

3. Pulsa **Test Connection**. Debe decir que la conexión fue exitosa. Si sale un error, ve a la sección 14.
4. Pulsa **OK** para guardar la conexión. Aparecerá como una tarjeta en Database Connections.

Por qué: Store Password guarda la contraseña de forma segura en tu usuario de Windows, así no la escribes cada vez que te conectas. Si prefieres que la pida siempre, no pulses Store Password: Workbench la pedirá al conectarte («Password will be prompted»). Test Connection comprueba los cinco datos antes de guardar, para no quedarte con una conexión que no funciona.

9. Paso 7: conectarte y ver las tablas

1. En la pantalla de inicio, haz clic en la tarjeta LeGrandHotel (local).
2. Se abre una pestaña nueva con el editor SQL, ya dentro de la base legrandhotel.
3. Abre la barra lateral (Ctrl + B si no se ve) y despliega la base legrandhotel → Tables para ver las 23 tablas.
4. Para ver el contenido de una tabla, escribe una consulta en el editor (sección 11) y ejecútala con Ctrl + Enter.

Estas son las tablas que debes ver (resultado real de SHOW TABLES):

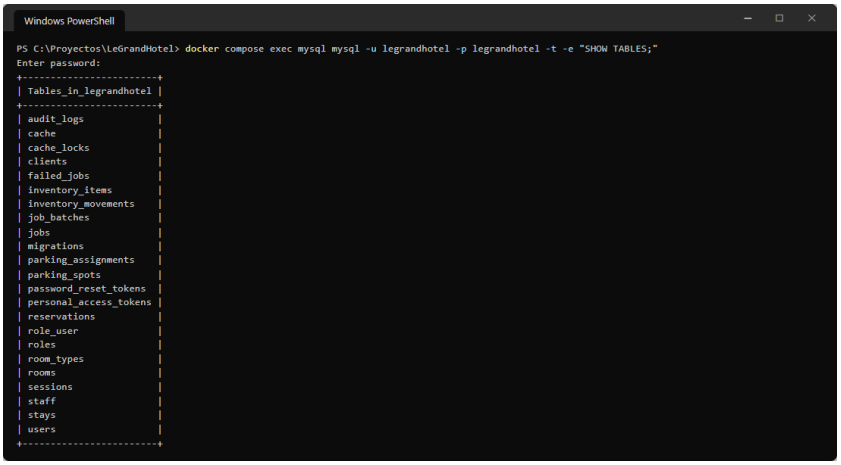


Figura 9. Las 23 tablas de la base legrandhotel.

10. Qué guarda cada tabla

10.1 Tablas del hotel

Tabla	Qué guarda	Se relaciona con
clients	Fichas de huéspedes: nombre, tipo y número de documento, teléfono y nacionalidad (el correo está en users). user_id indica si el huésped tiene cuenta web (rol Cliente)	users, reservations
room_types	Tipos de habitación: precio por noche, capacidad y amenities (lo que incluye, en formato JSON)	rooms

Tabla	Qué guarda	Se relaciona con
rooms	Habitaciones: número, piso, estado (available, occupied, cleaning, maintenance)	room_types, reservations
reservations	Reservas: huésped, habitación, fechas, estado y total	clients, rooms, stays
stays	Estadías: hora real de check-in y de check-out	reservations, parking_assignments
parking_spots	Espacios de la cochera y su estado	parking_assignments
parking_assignments	Qué vehículo (placa) ocupa qué espacio y desde cuándo	parking_spots, stays
inventory_items	Insumos del almacén: stock actual y mínimo	inventory_movements
inventory_movements	Entradas y salidas de cada insumo, con motivo y quién las hizo	inventory_items, users
staff	Personal del hotel: cargo, turno y fecha de ingreso	users
audit_logs	Auditoría: quién creó, editó o borró cada registro, con los valores antes y después	users

10.2 Cuentas y permisos

Tabla	Qué guarda
users	Cuentas: nombre, correo y contraseña cifrada (bcrypt). Los clientes tienen google_id y no tienen contraseña
roles	Los 5 roles: admin, receptionist, storekeeper, developer, client
role_user	Qué rol tiene cada cuenta (una cuenta puede tener varios)
personal_access_tokens	Tokens de sesión de la API (Sanctum), que vencen a los 60 minutos
password_reset_tokens	Pedidos de cambio de contraseña

10.3 Tablas internas de Laravel

Tabla	Qué guarda
migrations	Qué migraciones se aplicaron (es lo que lee migrate:status). No la edites
cache, cache_locks	Caché de la aplicación
sessions	Sesiones del navegador
jobs, job_batches, failed_jobs	Tareas en segundo plano (por ejemplo, correos) y las que fallaron

Por qué conviene saberlo: para revisar un dato casi siempre basta con las tablas del hotel. Las internas de Laravel las maneja el sistema solo y no hace falta abrirlas.

11. Consultas útiles (con su resultado real)

Copia cada consulta en el editor de Workbench y pulsa Ctrl + Enter. Debajo de cada una está lo que devolvió nuestra base el 27 de octubre de 2026.

11.1 Cuántos registros hay en cada tabla

SQL (en Workbench)

```
SELECT 'clients' AS tabla, COUNT(*) AS filas FROM clients
UNION ALL SELECT 'reservations', COUNT(*) FROM reservations
UNION ALL SELECT 'rooms', COUNT(*) FROM rooms
UNION ALL SELECT 'room_types', COUNT(*) FROM room_types
UNION ALL SELECT 'users', COUNT(*) FROM users
UNION ALL SELECT 'stays', COUNT(*) FROM stays
UNION ALL SELECT 'inventory_items', COUNT(*) FROM inventory_items;
```

Resultado real

```
+-----------------+-------+
| tabla           | filas |
+-----------------+-------+
| clients         |  1000 |
| reservations    |  5000 |
| rooms           |    70 |
| room_types      |     4 |
| users           |    55 |
| stays           |     1 |
| inventory_items |     6 |
+-----------------+-------+
```

Sirve para confirmar que los datos de demostración se cargaron. Los 55 usuarios son las 5 cuentas del equipo más los 50 recepcionistas de las pruebas k6.

11.2 Cuentas de demostración y su rol

SQL (en Workbench)

```
SELECT u.id, u.name, u.email, r.name AS rol
FROM users u
JOIN role_user ru ON ru.user_id = u.id
JOIN roles r ON r.id = ru.role_id
WHERE u.email LIKE '%@legrandhotel.test'
ORDER BY u.id;
```

Resultado real

```
+----+--------------------+----------------------------+--------------+
| id | name               | email                      | rol          |
+----+--------------------+----------------------------+--------------+
|  1 | Administrador Demo | admin@legrandhotel.test    | admin        |
|  2 | Recepción Demo     | recepcion@legrandhotel.test | receptionist |
|  3 | Almacén Demo       | almacen@legrandhotel.test  | storekeeper  |
|  4 | Programador Demo   | dev@legrandhotel.test      | developer    |
+----+--------------------+----------------------------+--------------+
```

Une tres tablas (JOIN): users tiene la cuenta, roles el nombre del rol y role_user dice qué rol tiene cada cuenta.

11.3 Tipos de habitación y lo que incluyen

SQL (en Workbench)

```
SELECT name, price_per_night, capacity, amenities FROM room_types;
```

Resultado real

```
+-------------+-----------------+----------+-------------------------------------------------------
-------------------------------+
| name        | price_per_night | capacity | amenities
|
+-------------+-----------------+----------+-------------------------------------------------------
-------------------------------+
| Simple      |          120.00 |        1 | ["Wi-Fi", "TV", "Baño privado", "Escritorio"]
|
| Doble       |          180.00 |        2 | ["Wi-Fi", "TV", "Baño privado", "Aire acondicionado"]
|
| Matrimonial |          220.00 |        2 | ["Wi-Fi", "Smart TV", "Baño privado", "Aire
acondicionado", "Vista a la ciudad"]       |
| Suite       |          450.00 |        4 | ["Wi-Fi", "Smart TV", "Jacuzzi", "Minibar", "Desayuno
incluido", "Sala independiente"] |
+-------------+-----------------+----------+-------------------------------------------------------
-------------------------------+
```

amenities es una columna JSON: guarda la lista de servicios que ve el cliente al reservar.

11.4 Próximas llegadas

SQL (en Workbench)

```
SELECT r.id, c.full_name AS huesped, ro.number AS hab,
       r.check_in, r.check_out, r.status, r.total
FROM reservations r
JOIN clients c ON c.id = r.client_id
JOIN rooms ro ON ro.id = r.room_id
WHERE r.check_in >= CURDATE()
ORDER BY r.check_in
LIMIT 8;
```

Resultado real

```
+------+-----------------------------------------+------+------------+------------+-----------+----
----+
| id   | huesped                                 | hab  | check_in   | check_out  | status    |
total  |
+------+-----------------------------------------+------+------------+------------+-----------+----
----+
|  798 | Juana María Paula Lozano Montaño Hijo   | 1007 | 2026-10-28 | 2026-10-29 | confirmed |
450.00 |
| 1697 | Nahuel Eduardo Naranjo Baca             | 1016 | 2026-10-28 | 2026-10-29 | confirmed |
120.00 |
| 2497 | Dn. Thiago Alarcón Figueroa Hijo        | 1024 | 2026-10-28 | 2026-10-29 | confirmed |
120.00 |
| 1499 | Sophie Josefina Venegas Santillán Hijo  | 1014 | 2026-10-28 | 2026-10-30 | confirmed |
440.00 |
| 4998 | Magdalena Gabriela Cabrera Cantú Hijo   | 1049 | 2026-10-28 | 2026-10-30 | confirmed |
360.00 |
|  995 | Valentina Guadalupe Delgadillo Montañez | 1009 | 2026-10-28 | 2026-10-31 | confirmed |
540.00 |
| 1797 | Santiago Javier Serna Páez              | 1017 | 2026-10-28 | 2026-10-31 | confirmed |
540.00 |
| 2697 | Dn. Francisco Alicea Palomo             | 1026 | 2026-10-28 | 2026-10-31 | confirmed |
660.00 |
+------+-----------------------------------------+------+------------+------------+-----------+----
----+
```

CURDATE() es la fecha de hoy; LIMIT 8 muestra solo las primeras 8 para no cargar las 5000.

11.5 Reservas por estado

SQL (en Workbench)

```
SELECT status, COUNT(*) AS cantidad FROM reservations GROUP BY status;
```

Resultado real

```
+------------+----------+
| status     | cantidad |
+------------+----------+
| completed  |     4928 |
| confirmed  |       71 |
| checked_in |        1 |
+------------+----------+
```

GROUP BY junta las reservas por estado y COUNT(*) las cuenta. completed = ya terminaron; confirmed = por llegar; checked_in = el huésped está dentro.

12. Cuidado al modificar datos

Workbench permite editar y borrar filas directamente, pero en LeGrandHotel conviene hacerlo desde la página y no desde la base. Estas son las razones:

Si cambias algo en Workbench…	Lo que se pierde
Las validaciones	La API revisa fechas, capacidad, stock y documentos repetidos. En la base puedes crear una reserva que se cruza con otra o dejar stock negativo
La auditoría	Cada cambio hecho desde la página queda en audit_logs con quién y cuándo. Un cambio en Workbench no deja rastro
Las reglas de negocio	Un check-out desde la página libera la habitación y la cochera. Cambiar solo el estado en la base deja esos datos inconsistentes
Las contraseñas	Están cifradas con bcrypt. Si escribes una contraseña en texto plano en users.password, esa cuenta no podrá entrar

Regla práctica: usa Workbench para mirar (SELECT). Si de verdad necesitas cambiar algo, primero haz un respaldo (sección 13) y escribe la consulta con WHERE y el id exacto. Un UPDATE o DELETE sin WHERE cambia o borra la tabla entera.

13. Respaldo y apagado

13.1 Hacer un respaldo

Desde la carpeta del proyecto, en PowerShell (te pedirá la contraseña):

PowerShell

```
docker compose exec mysql sh -c "mysqldump -u legrandhotel -p legrandhotel > /tmp/respaldo.sql"
docker compose cp mysql:/tmp/respaldo.sql .\respaldo-legrandhotel.sql
```

Por qué: mysqldump copia toda la base (estructura y datos) en un archivo .sql. Se genera dentro del contenedor y luego se copia afuera con docker compose cp porque, si se guardara directo con > en PowerShell, el archivo quedaría en otra codificación (UTF-16) y MySQL no lo podría leer al restaurarlo. No subas ese archivo a GitHub: tiene datos y contraseñas cifradas.

13.2 Apagar la base

PowerShell

```
docker compose stop mysql
```

Por qué: apaga el contenedor y libera memoria. Los datos siguen guardados en el volumen mysql-data: la próxima vez que hagas docker compose up -d mysql estará todo igual.

Nunca uses docker compose down -v si quieres conservar los datos: la opción -v borra el volumen, es decir, la base completa.

14. Si algo sale mal

Lo que ves	Por qué pasa	Qué hacer
«Can't connect to MySQL server on 127.0.0.1:3306»	Docker o el contenedor está apagado	Pasos 1 y 2; espera a que diga (healthy)
«Access denied for user 'legrandhotel'»	Contraseña distinta de la de DB_PASSWORD	Vuelve a pulsar Store Password y cópiala exacta de api\.env
Access denied aunque la contraseña es la de api\.env	La base se creó con otra MYSQL_PASSWORD (el volumen guarda la primera)	Usa la contraseña con la que se creó; pregunta al equipo antes de borrar el volumen
docker compose up falla: «port 3306 is already allocated»	Otro MySQL usa el puerto, por ejemplo el de XAMPP	Detén MySQL en el panel de XAMPP (o el servicio MySQL de Windows) y repite el paso 2
Workbench conecta, pero no ve las tablas	Te conectaste a otro MySQL (el de XAMPP) o la base está vacía	Comprueba el paso 2 y corre las migraciones (paso 3)
«Unknown column 'user_id'» o falta amenities	Falta una migración	php artisan migrate (paso 3)
La ventana de Workbench se queda en negro	La interfaz nueva está cargando	Espera unos segundos o pulsa Ctrl + B
«docker» no se reconoce como comando	Docker Desktop no está instalado o no se abrió nunca	Instálalo, ábrelo una vez y abre una terminal nueva

15. Sin Workbench: consultar desde la terminal

Si no tienes Workbench a la mano, el contenedor trae el cliente de MySQL. Desde la carpeta del proyecto:

PowerShell

```
docker compose exec mysql mysql -u legrandhotel -p legrandhotel -t -e "SELECT
status, COUNT(*) AS cantidad FROM reservations GROUP BY status;"
```

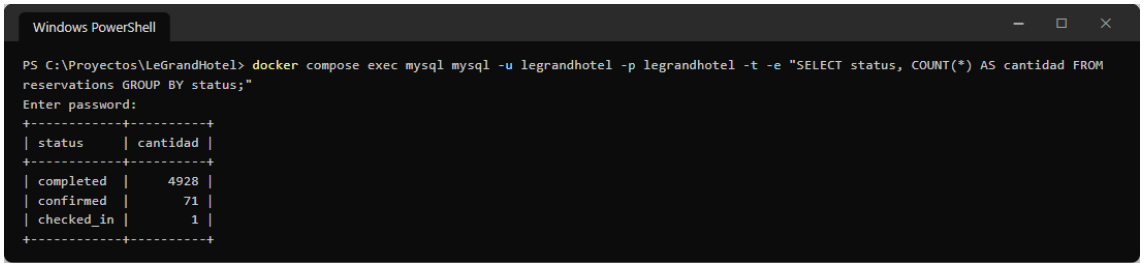


Figura 10. La misma consulta de la sección 11.5, desde la terminal.

-p hace que pida la contraseña (no se ve al escribirla), legrandhotel es la base, -t dibuja la tabla con bordes y -e ejecuta la consulta entre comillas. Sin -e entras al modo interactivo (mysql>), del que sales con exit.

16. Resumen rápido

PowerShell

```
# 1. Encender la base (con Docker Desktop abierto)
cd C:\Proyectos\LeGrandHotel
docker compose up -d mysql
docker compose ps mysql            # debe decir (healthy)

# 2. Tablas al día
cd api
php artisan migrate

# 3. Workbench: + -> Host 127.0.0.1, Port 3306, User legrandhotel,
#    Default Schema legrandhotel, Store Password (DB_PASSWORD de api\.env)

# 4. Al terminar
cd ..
docker compose stop mysql
```